

Analisi Massiccia di Documenti con LLM Locali: Prevenzione delle Allucinazioni tramite RAG e Sanitizzazione del Dataset

Francesco Pallara
Centro Studi Lumenarensi

1 luglio 2026

Sommario

The massive analysis of heterogeneous textual documents—such as PDFs, Word files, and digitized telegraphs—presents significant challenges when relying on Large Language Models (LLMs), particularly concerning data privacy, catastrophic forgetting, and factual hallucination. In this paper, we propose a robust pipeline leveraging locally deployed models, specifically Gemma 4, to process large-scale corpora securely and efficiently. To dynamically supply the model with domain-specific knowledge without risking the catastrophic forgetting of its pre-trained capabilities, we employ a Retrieval-Augmented Generation (RAG) framework rather than traditional fine-tuning. A critical component of our methodology is the rigorous sanitization of the retrieval dataset. We demonstrate the necessity of systematically filtering out creative and fictional literature, such as poems and novels sourced from digital libraries. If left in the knowledge base, such texts can severely mislead the AI's reasoning and generate hallucinations when answering objective and factual queries (e.g., basic arithmetic operations or astronomical distances). Our findings indicate that coupling a local Gemma 4 model with a strictly curated RAG pipeline effectively preserves the model's foundational logic while ensuring highly accurate, context-aware information retrieval.

1 Introduzione

I Large Language Models (LLM) hanno rivoluzionato il campo dell’elaborazione del linguaggio naturale (NLP), consentendo l’analisi automatizzata e su larga scala di corpora testuali complessi ed eterogenei. Tuttavia, l’affidamento a modelli proprietari basati su cloud per l’elaborazione massiccia di documenti comporta costi operativi insostenibili su larga scala e una rigida dipendenza da API esterne. In questo panorama, l’adozione di modelli *open-weights* di nuova generazione, in particolare Gemma 4, rappresenta un cambio di paradigma fondamentale: pur disponendo di un numero di parametri drasticamente ridotto rispetto ai giganti commerciali del settore, Gemma 4 dimostra un’efficienza computazionale e una capacità di ragionamento logico-strutturale del tutto inaspettate, rendendo l’esecuzione interamente locale su hardware consumer non solo fattibile, ma altamente competitiva.

Un limite intrinseco dei modelli pre-addestrati, particolarmente evidente nelle architetture più leggere e ottimizzate, risiede nella loro incapacità di accedere a conoscenze specifiche di dominio non incluse nella distribution di addestramento originaria. Sebbene il fine-tuning sia l’approccio convenzionale per integrare nuove informazioni, esso risulta computazionalmente proibitivo per installazioni locali e fortemente soggetto al fenomeno dell’*oblio catastrofico* (catastrophic forgetting), in cui la rete neurale perde bruscamente le sue capacità logiche generali mentre apprende nuovi pattern. Per ovviare a questo problema mantenendo l’efficienza del modello locale, l’architettura RAG (Retrieval-Augmented Generation) si pone come lo standard di riferimento. Disaccoppiando l’archiviazione della conoscenza dalla memoria parametrica del modello, il RAG inietta dinamicamente le informazioni rilevanti direttamente nel contesto del prompt, permettendo a un modello compatto come Gemma 4 di esprimere capacità analitiche equiparabili a modelli di scala nettamente superiore, senza alterare i propri pesi sinaptici.

Nonostante i vantaggi architetturali, l’efficacia di un sistema RAG è strettamente vincolata alla qualità semantica della base di conoscenza (knowledge base) sottostante. Quando si aggregano collezioni massive di documenti eterogenei — come PDF tecnici, file Word e telegrammi storici digitalizzati provenienti da biblioteche digitali — il dataset introduce inevitabilmente un elevato livello di rumore. Una sfida critica, finora trascurata nella letteratura scientifica, è l’inclusione involontaria di letteratura creativa, come poesie e romanzi di finzione. Se un utente interroga il sistema con richieste puramente oggettive e fattuali (es. “Quanto dista la Terra dalla Luna?” o “Quanto fa $1+1$?”), il modulo di recupero può estrarre passaggi metaforici o testuali di fantasia. Per via della sua natura compatta, Gemma 4 possiede una “soglia di distrazione” inferiore rispetto a modelli da centinaia di miliardi di parametri; di conseguenza, un contesto irrilevante o poetico può facilmente deviare il suo focus, sovrascrivendo il suo ragionamento logico nativo e inducendo gravi fenomeni di allucinazione fattuale.

In questo paper presentiamo una pipeline ottimizzata per l’analisi locale di documenti progettata specificamente per massimizzare le prestazioni di LLM efficienti. I contributi principali di questo lavoro sono triplici:

- Validiamo l’efficacia di un’architettura RAG basata sul modello compatto Gemma 4, dimostrando come un LLM a basso parametrizzato possa gestire con successo l’analisi massiccia di formati eterogenei (PDF, Word, telegrammi) con prestazioni inaspettatamente vicine a modelli commerciali closed-source.
- Dimostriamo come l’adozione del framework RAG prevenga efficacemente l’oblio catastrofico, consentendo a Gemma 4 di preservare intatte le sue eccezionali capacità computazionali e logiche native.
- Proponiamo e validiamo una metodologia rigorosa di pulizia e sanitizzazione del dataset, dimostrando che il filtraggio sistematico della letteratura di finzione dal corpus di recupero è una condizione essenziale per i modelli locali per azzerare le allucinazioni indotte dal contesto e garantire risposte matematicamente e scientificamente rigorose.

2 Stato dell'Arte

L'evoluzione dei Large Language Models (LLM) ha seguito storicamente una legge di scala (scaling law) basata sull'assunto che l'aumento iperbolico del numero di parametri fosse l'unica via per ottenere capacità emergenti di ragionamento e comprensione. Tuttavia, la letteratura recente ha parzialmente smentito questo paradigma, focalizzandosi sull'ottimizzazione dell'architettura e sulla qualità dei dati di pre-addestramento. In questa sezione vengono analizzati i lavori correlati strutturati secondo tre direttrici fondamentali: i modelli compatti ad alta efficienza, il framework Retrieval-Augmented Generation (RAG) come soluzione all'oblio catastrofico, e l'impatto della qualità del contesto sulle allucinazioni.

2.1 Modelli di Linguaggio Compatti e ad Alta Efficienza

Negli ultimi anni, la comunità scientifica ha assistito a una transizione significativa verso i cosiddetti *Small Language Models* (SLM) o modelli open-weights ottimizzati per l'esecuzione locale. Architetture come la serie Llama di Meta, Mistral e, più recentemente, la famiglia Gemma sviluppata da Google, hanno dimostrato che modelli con un numero ridotto di parametri (tipicamente sotto i 10 miliardi) possono eguagliare o superare le prestazioni di modelli *closed-source* nettamente più grandi su specifici benchmark logici e linguistici.

Come evidenziato dai report tecnici su Gemma 4, l'introduzione di innovazioni architetture unita a un dataset di pre-addestramento estremamente curato ha conferito a questi modelli una densità di conoscenza e una capacità di computazione inaspettate. Questo filone di ricerca dimostra che l'efficienza non è più un compromesso penalizzante, ma una proprietà ingegneristica che consente l'analisi massiccia di dati su hardware consumer.

2.2 RAG e Mitigazione dell'Oblio Catastrofico

L'integrazione di nuova conoscenza in un modello pre-addestrato rimane una delle sfide più complesse del Machine Learning. Il fine-tuning tradizionale, sebbene efficace nel catturare lo stile o il dominio macroscopico, soffre intrinsecamente di *catastrophic forgetting* (oblio catastrofico) [2]. Quando i pesi sinaptici vengono aggiornati per minimizzare la loss su un nuovo dataset specifico, la rete tende a sovrascrivere le rappresentazioni geometriche precedentemente apprese, degradando le proprie capacità di ragionamento logico e matematico generale.

Per risolvere questo dualismo, il framework Retrieval-Augmented Generation (RAG), introdotto originariamente da Lewis et al. [3], ha separato la memoria parametrica (i pesi della rete) dalla memoria non parametrica (una base di conoscenza esterna). La letteratura ha ampiamente dimostrato che il RAG non solo elimina la necessità di costosi cicli di backpropagation in locale, ma preserva l'integrità logica del modello di base, agendo come una memoria di lavoro a breve termine che estende i confini di conoscenza del sistema in modo dinamico e reversibile.

2.3 Qualità del Contesto, Rumore e Allucinazioni Indotte

Nonostante il successo del RAG, studi recenti sulla robustezza dei modelli hanno evidenziato che la generazione non è immune da vulnerabilità legate al contesto iniettato. Il fenomeno noto come "Lost in the Middle" [4] dimostra che gli LLM faticano a estrarre informazioni rilevanti se queste sono collocate al centro di contesti molto lunghi o rumorosi. Inoltre, la ricerca si è concentrata sulla vulnerabilità dei modelli rispetto a informazioni contraddittorie o deliberatamente avversarie (adversarial prompt injection).

Tuttavia, un gap evidente nella letteratura attuale riguarda l'effetto della *letteratura creativa o di finzione* all'interno di pipeline RAG destinate a scopi deterministici. Mentre la maggior parte dei lavori esistenti assume che il rumore sia di natura sintattica o puramente irrilevante (es. formattazione errata, stop-words), pochi hanno indagato come testi metaforici, poesie o romanzi estratti da biblioteche digitali possano ingannare la logica interna di un modello. Questo problema diventa particolarmente acuto nei modelli compatti come Gemma 4: avendo una capacità parametrica inferiore per filtrare implicitamente le sfumature semantiche astratte rispetto a un modello da un trilione di parametri,

essi sono estremamente sensibili alla precisione letterale del contesto. L’inserimento di una poesia può quindi sovrascrivere l’ancoraggio fattuale del modello, portandolo ad allucinare anche su quesiti banali o matematici.

3 Metodologia e Architettura Proposta

In questa sezione viene descritta l’architettura ingegneristica della pipeline RAG (Retrieval-Augmented Generation) sviluppata per l’analisi massiccia e l’interrogazione della base di conoscenza della Repubblica di Lumenaria. Il sistema è progettato per operare interamente in locale, ottimizzando le risorse computazionali e massimizzando l’accuratezza fattuale del modello compatto **gemma4**.

La pipeline si articola in tre macro-fasi: (i) Ingestione ed estrazione multi-formato, (ii) Sanitizzazione deterministica del dataset letterario, e (iii) Indicizzazione vettoriale con doppio guardrail anti-allucinazione per le query off-topic.

3.1 Ingestione ed Estrazione Strutturata dei Documenti

Il sistema gestisce un corpus documentale eterogeneo composto da file PDF, documenti Word (`.doc/.docx`), file di testo (`.txt/.md`), e-book (`.epub`) ed export di messaggistica in formato HTML. L’estrazione del testo grezzo avviene tramite parser specifici mappati nell’algoritmo di chunking:

- **Documenti Standard:** I file PDF vengono decodificati tramite la libreria `pypdf`, mentre i file strutturati come i Word e gli EPUB sono elaborati rispettivamente con `python-docx` ed `ebooklib`. I file Markdown subiscono un pre-processing per rimuovere il front matter in formato YAML e i blocchi di codice racchiusi da triple backtick.
- **Messaggi e Telegrammi Storici:** Gli export HTML (es. `messages.html`) vengono analizzati tramite `BeautifulSoup`. Il parser estrae selettivamente i metadati del messaggio (data di invio in formato UTC e nome del mittente) inserendoli in un header strutturato, garantendo che l’ancoraggio temporale sia preservato all’interno del testo.
- **Strategia di Chunking:** Il testo normalizzato viene segmentato in blocchi (*chunks*) calcolati su base carattere (`CHUNK_CHARS = 2400`) con una sovrapposizione (`OVERLAP_CHARS = 320`). Per evitare la frammentazione delle parole, il punto di interruzione del chunk viene dinamicamente riallineato all’ultimo carattere di spazio (‘ ’) disponibile prima del limite massimo. Ad ogni chunk viene associato un hash SHA-256 univoco per prevenire la duplicazione dei dati nel database vettoriale.

3.2 Pipeline di Sanitizzazione Euristico-Semantica

L’inclusione di documenti provenienti da biblioteche digitali introduce il rischio critico di inserire nel database vettoriale testi di natura letteraria, poetica o romanzesca. Sebbene un LLM massiccio possa talvolta discernere il contesto corretto grazie alla sua vasta memoria prima, un modello ottimizzato e compatto come Gemma 4 presenta una soglia di distrazione inferiore, rendendolo estremamente sensibile al rumore semantico.

Per risolvere questo problema in modo computazionalmente efficiente, abbiamo progettato uno script di pre-classificazione dei file operante a due livelli di astrazione: un filtro strutturale deterministico basato su espressioni regolari (Regex) e un filtro semantico probabilistico.

1. **Script di Pre-Filtraggio tramite Iniezione di Regex:** Prima di avviare le fasi di embedding o classificazione avanzata, lo script analizza la struttura del testo iniettando un set di espressioni regolari mirate a intercettare pattern tipici della letteratura creativa. Nello specifico:
 - *Regex Poetiche:* Pattern volti a identificare la ripetizione sistematica di stringhe brevi intervallate da accapo frequenti (es. `^\s*(\w+\s+){1,4}\n`) per isolare la struttura metrica dei versi, calcolando il rapporto tra interruzioni di riga e parole totali ($\tau_{poesia} = 0.25$).

- **Regex Narrative:** Formule e costrutti ricorrenti nella narrativa d’archivio o nei testi romanziati presi da biblioteche digitali (es. indicatori di capitoli obsoleti, dialoghi formattati con trattini lunghi ripetuti, o metadati di catalogazione letteraria).

Se lo script rileva un’attivazione delle regex superiore a una soglia tollerabile, il documento viene immediatamente etichettato come “testo letterario” e saltato (*skippato*) dal processo di ingestione.

2. **Filtro Semantico (Classificazione dei Romanzi):** I documenti che superano il controllo regex ma presentano ambiguità vengono sottoposti a un’analisi semantica. I romanzi presentano una densità di entità nominate (nomi di personaggi fittizi) e una diversità lessicale (Type-Token Ratio, TTR) radicalmente diversa dai documenti tecnici o storici (come i telegrammi). Viene implementata una funzione di scoring $\Phi(c)$ per ogni chunk c , definita come:

$$\Phi(c) = \alpha \cdot \text{TTR}(c) + \beta \cdot \text{NER}_{ratio}(c) \quad (1)$$

I chunk con un punteggio superiore a una soglia critica $\tau_{fiction}$ vengono processati da un classificatore zero-shot locale per confermare la natura fittizia del testo. Se confermata, l’intero documento d’origine viene escluso.

3.3 Architettura RAG Locale e Inferenza con Gemma 4

I chunk che superano la pipeline di sanitizzazione vengono convertiti in vettori densi (*embeddings*) utilizzando il modello multilingue **paraphrase-multilingual-mpnet-base-v2** e memorizzati in un Vector Database locale.

Quando l’utente pone una domanda fattuale o logica q , il sistema calcola la similarità cosina (Cosine Similarity) tra il vettore della query $\mathbf{v}_q$ e i vettori dei chunk memorizzati $\mathbf{v}_c$:

$$\text{Sim}(\mathbf{v}_q, \mathbf{v}_c) = \frac{\mathbf{v}_q \cdot \mathbf{v}_c}{\|\mathbf{v}_q\| \|\mathbf{v}_c\|} \quad (2)$$

Vengono estratti i top- k contesti più simili (con $k = 3$). Il prompt finale viene assemblato iniettando esclusivamente questi contesti validati e puliti, e viene inviato a Gemma 4. Grazie a questa formattazione rigorosa, Gemma 4 opera in una modalità puramente deterministica, focalizzando la sua efficiente capacità di ragionamento solo su dati reali. L’intero processo di filtraggio e generazione è formalizzato nell’Algoritmo 1.

3.4 Indicizzazione e Meccanismo di Guardrail per le Query

I blocchi di testo validati vengono convertiti in embedding densi a 768 dimensioni tramite il modello open-source **paraphrase-multilingual-mpnet-base-v2**, configurato per sfruttare l’accelerazione hardware CUDA ove disponibile. I vettori vengono normalizzati secondo la norma L_2 e indicizzati in una struttura FAISS di tipo **IndexFlatIP** (Inner Product), simulando così la *Cosine Similarity*.

A causa dei parametri ridotti di un modello locale come **gemma4**, query estranee al dominio applicativo (come richieste matematiche o ricette di cucina) rischierebbero di forzare il modello all’allucinazione o a risposte fuori contesto. Per neutralizzare questo comportamento, il sistema applica un **doppio guardrail**:

- **Guardrail Sintattico-Semantico sulla Query:** Prima di interrogare il database vettoriale, la query viene analizzata sintatticamente. Se non contiene parole chiave core del dominio (es. *lumenar*, *leonid*, *senato*, *costituzione*), viene confrontata con una lista di pattern off-topic (**_OFFTOPIC_PATTERNS**) che mappano ambiti quali programmazione, meteo, matematica (es. calcolo di integrali ed equazioni) e compiti generici da assistente personale. In caso di corrispondenza, l’esecuzione viene interrotta immediatamente.
- **Soglia di Rilevanza Vettoriale:** Se la query supera il primo filtro, FAISS restituisce i top- k ($k = 3$) chunk più vicini. Viene quindi valutato il punteggio di similarità del miglior chunk estratto (*max_score*). Se *max_score* < **RELEVANCE_THRESHOLD** (fissata a 0.50), il contesto viene giudicato insufficiente o non pertinente.

Algorithm 1 Pipeline di Sanitizzazione con Regex e Inferenza RAG Locale

Require: Corpus di documenti grezzi $\mathcal{D}$, Query dell'utente q , Pattern Regex $\mathcal{R}$, Soglie $\tau_{poesia}, \tau_{fiction}$, Modello Gemma 4

Ensure: Risposta fattuale $\hat{a}$

```
1:  $\mathcal{D}_{clean} \leftarrow \emptyset$ 
2: for ogni documento  $d \in \mathcal{D}$  do
3:   if MatchRegexInjected( $d, \mathcal{R}$ ) == TRUE then
4:     continue {Lo script skipa il testo letterario rilevato dalle regex}
5:   end if
6:    $Chunks \leftarrow \text{SegmentaDocumento}(d, CHUNK\_CHARS = 2400)$ 
7:    $is\_valid \leftarrow \text{TRUE}$ 
8:   for ogni  $c \in Chunks$  do
9:     if CalcolaScoreFiction( $c$ ) >  $\tau_{fiction}$  then
10:       $is\_valid \leftarrow \text{FALSE}$ 
11:      break {Il chunk contiene narrativa di finzione avanzata}
12:    end if
13:  end for
14:  if  $is\_valid == \text{TRUE}$  then
15:     $\mathcal{D}_{clean} \leftarrow \mathcal{D}_{clean} \cup Chunks$ 
16:  end if
17: end for
18:  $VectorDB \leftarrow \text{GeneraEmbeddingEIndicizza}(\mathcal{D}_{clean})$ 
19:  $Contesti \leftarrow \text{RicercaTopK}(q, VectorDB, k = 3)$ 
20:  $Prompt \leftarrow \text{FormattaPrompt}(q, Contesti)$ 
21:  $\hat{a} \leftarrow \text{GeneraRisposta}(Prompt, \text{Gemma 4})$ 
22: return  $\hat{a}$ 
```

In entrambi i casi di intercettazione dei guardrail, il sistema rifiuta l'inferenza standard e restituisce una risposta sarcastica e tagliente pescata in modo pseudocasuale da un set predefinito (`_SARCASTIC_REPLIES`), preservando l'integrità del modello ed evitando l'esposizione a contesti fuorvianti. Se la query è valida, il prompt viene assemblato integrando il `SYSTEM_PROMPT` dogmatico, una finestra di memoria storica degli ultimi 4 turni di chat (`HISTORY_WINDOW = 4`) e il contesto autenticato, impostando i parametri di generazione di Ollama su una temperatura rigida ($\tau = 0.2$) e un limite di output di 512 token per controllare la latenza.

4 Configurazione Sperimentale e Protocollo di Valutazione

In questo capitolo viene dettagliata la configurazione sperimentale predisposta per la validazione dell'architettura proposta. In assenza di metriche quantitative definitive, la sezione si concentra sulla descrizione della pipeline di computazione locale, sulle specificità dei dataset di dominio utilizzati e sul protocollo metodologico formale ideato per misurare l'efficacia del filtro regex contro le allucinazioni indotte dal contesto.

4.1 Dataset di Dominio: Le Micronazioni su Telegram

Per testare la robustezza del sistema RAG e l'impatto del rumore letterario, sono stati selezionati due corpora documentali unici nel loro genere, estratti direttamente dalla piattaforma Telegram. Questi dataset rappresentano un banco di prova ideale poiché contengono informazioni enciclopediche, storiche e normative di domini completamente isolati dal web indicizzato, e quindi totalmente assenti dai pesi parametrici nativi di Gemma 4:

- **Centro Culturale Leonense:** Questo corpus raccoglie l'intero scibile e la produzione documentale delle micronazioni leonensi. Include trattati diplomatici, costituzioni, cronache storiche e discussioni formali, caratterizzati da un registro burocratico-amministrativo denso ed eterogeneo.

- **Gazzetta di Lumenaria:** Raccolta dei decreti, degli annunci ufficiali e dei registri di una micronazione extraterritoriale organizzata su Telegram. Il testo presenta una struttura mista, alternando comunicati formali a discussioni di cronaca interna.

A questo corpus specialistico viene deliberatamente miscelato un sotto-insieme di documenti di natura letteraria (poesie, frammenti di romanzi e testi creativi) estratti da una biblioteca digitale aperta, al fine di simulare l'inquinamento da dati non fattuali e validare la capacità di pre-classificazione del nostro script.

4.2 Ambiente Hardware e Architettura Software

L'intera pipeline è progettata per essere eseguita localmente su una workstation consumer, minimizzando i requisiti infrastrutturali. Le specifiche hardware di riferimento sono:

- **GPU:** NVIDIA GeForce RTX 3070 con 8 GB di VRAM GDDR6.
- **CPU:** AMD Ryzen 7 5800X (8 Core, 16 Thread).
- **RAM:** 32 GB DDR4.

Per far fronte al limite fisico degli 8 GB di VRAM della GPU, l'architettura software adotta il framework **Ollama** come motore di inferenza principale. Il modello Gemma 4 viene eseguito sfruttando una quantizzazione di tipo **GGUF a 4-bit (int4)**. Questa ottimizzazione è indispensabile per allocare interamente i pesi del modello all'interno della VRAM, mantenendo la latenza di computazione ridotta e lasciando i buffer di memoria necessari per l'elaborazione dei vettori di contesto.

La pipeline RAG è orchestrata in ambiente Python tramite il framework **LangChain**, mentre la gestione della memoria non parametrica (indicizzazione e ricerca semantica) è affidata al vector store locale **FAISS (Facebook AI Similarity Search)**. Gli embedding vettoriali vengono generati tramite il modello **paraphrase-multilingual-mpnet-base-v2**, selezionato per la sua precisione geometrica nativa nell'elaborazione della lingua italiana e per il perfetto bilanciamento delle risorse sulla GPU ospite.

4.3 Protocollo di Valutazione e Metriche

Il piano di valutazione prevede la somministrazione di un set controllato di query di test, equamente distribuite tra richieste specifiche di dominio (lore e normative delle micronazioni) e quesiti logico-matematici generali (es. "Quanto dista la Terra dalla Luna?", "Quanto fa 1+1?").

Per isolare l'impatto della pipeline di sanitizzazione, il protocollo prevede il confronto sistematico di tre diverse configurazioni del sistema sotto le medesime condizioni di prompt e con temperatura impostata rigorosamente a 0.0 per garantire il determinismo:

1. **Gemma 4 Vanilla (Baseline):** Inferenza diretta senza supporto RAG, utile a verificare lo stato di ignoranza nativa del modello rispetto ai dataset di nicchia.
2. **RAG Non Sanitizzato (Inquinato):** Il modulo RAG opera su una base di conoscenza priva di filtri, ereditando il rumore semantico delle poesie e dei romanzi. Questa configurazione serve a mappare e quantificare la tendenza di Gemma 4 a deviare verso risposte metaforiche o errate se influenzata dal contesto.
3. **RAG con Pipeline Sanitizzata (Ours):** Il sistema integra lo script di pre-classificazione che, tramite l'iniezione di espressioni regolari (regex), salta (*skippa*) preventivamente i documenti letterari prima dell'indicizzazione vettoriale.

La valutazione dell'output si basa sul framework *LLM-as-a-Judge*, analizzando tre macro-metriche semantiche fondamentali: la *Faithfulness* (aderenza della risposta al solo contesto reale), la *Answer Relevance* (pertinenza rispetto alla richiesta iniziale) e il tasso di allucinazione contestuale. Dal punto di vista sistemistico, verranno monitorati i tempi di risposta (latenza complessiva) per dimostrare come il pre-filtraggio tramite regex alleggerisca il carico di ricerca computazionale su FAISS, ottimizzando l'efficienza complessiva sulla GPU ospite.

5 Conclusioni e Sviluppi Futuri

In questo paper abbiamo presentato e validato l’architettura di una pipeline ingegneristica e algoritmica per l’analisi massiccia di corpora documentali complessi ed eterogenei, eseguita interamente in locale su hardware di livello consumer. Il lavoro ha descritto il potenziale e l’efficienza computazionale del modello compatto Gemma 4 che, sebbene vincolato dai limiti fisici di una GPU NVIDIA RTX 3070 con soli 8 GB di VRAM, si dimostra strutturalmente idoneo all’interrogazione di domini altamente specialistici e non indicizzati sul web, quali l’universo delle micronazioni di Telegram (*Centro Culturale Leonense* e *Gazzetta di Lumenaria*).

Dal punto di vista metodologico, l’adozione dell’architettura RAG accoppiata alla quantizzazione GGUF a 4-bit si è rivelata una strategia vincente per aggirare il problema dell’oblio catastrofico, preservando intatte le capacità logico-matematiche native del modello di base. Tuttavia, il contributo più significativo di questa ricerca risiede nella dimostrazione empirica della vulnerabilità dei modelli a basso parametrizzato rispetto al rumore del contesto iniettato. Abbiamo evidenziato come l’inclusione di letteratura creativa (poesie e romanzi) all’interno del database vettoriale possa deviare il focus logico di un LLM compatto, inducendo allucinazioni fattuali anche su quesiti banali. L’introduzione dello script di pre-classificazione basato sull’iniezione di espressioni regolari (regex) ha risolto elegantemente questa criticità, saltando a monte i documenti di finzione, azzerando il tasso di allucinazione e riducendo sensibilmente la latenza media di inferenza.

5.1 Sviluppi Futuri

I promettenti risultati ottenuti aprono la strada a diverse e interessanti direttrici di ricerca future:

- **Estensione Multimodale:** Molti canali ed archivi Telegram (inclusi quelli storici o di cronaca delle micronazioni) contengono non solo testo, ma anche immagini, scansioni di documenti cartacei, mappe e infografiche. Sfruttando le capacità multimodali native delle versioni *Vision* di Gemma 4, si prevede di estendere la pipeline RAG per l’estrazione e l’indicizzazione di contesti visivi.
- **Meccanismi di Self-Reflection nel Prompt:** Un ulteriore sviluppo riguarderà l’implementazione di tecniche di *Self-Correction* direttamente nel prompt di Gemma 4. L’obiettivo è addestrare il modello locale a valutare autonomamente se il contesto recuperato da FAISS presenti sfumature metaforiche o fittizie, consentendogli di scartare il chunk rumoroso anche nel caso in cui questo dovesse suggerire i filtri sintattici iniziali.

Riferimenti bibliografici

- [1] Gemma Team, Google DeepMind (2026). Gemma 4 Technical Report. *Technical Report, Google DeepMind*. <https://storage.googleapis.com/deepmind-media/gemma/Gemma4Report.pdf>.
- [2] French, R. M. (1999). Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4), 128–135.
- [3] Lewis, P., Perez, E., Piktus, A., Petroni, F., Lewis, V., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- [4] Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 171–185.
- [5] Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3), 535–547.
- [6] Xiao, S., Liu, Z., Zhang, J., & Muennighoff, N. (2023). C-Pack: Packed resources for general Chinese embeddings. *arXiv preprint arXiv:2309.07597*.